

私人工程教程 • 第三冊

AI 交易動手實作課

一步步拆解一個「真的能跑」的交易閉環

訊號 → 回測 → 驗證 → 風控 → paper 下單

搭配專案：`~/Delvin-agent/ai_trading_course/demo/`（可直接 `python3 run.py`）

對象：讀完《AI 交易工程地圖》後，要親手把它跑起來的你

立場：誠實第一 • 這個 demo 的目的不是賺錢，是讓你親眼看到「驗證怎麼殺掉漂亮的假策略」

完成日：2026 年 6 月

老師的話：這次我們動手

《地圖》給你全貌，這本帶你動手。我已經幫你把一個**真的能跑**的最小閉環寫好放在你電腦裡了。這本書的每一章，對應 demo 裡的一個檔案——你可以一邊讀、一邊打開那個檔對照、一邊改。

先把它跑起來（30 秒）：

```
cd ~/Delvin-agent/ai_trading_course/demo
python3 run.py
```

第一次會抓 FinMind 台股資料並快取，之後秒跑。看到一堆數字後，再回來讀這本書，你會懂每個數字怎麼來、為什麼重要。

這個 demo 把《地圖》第一部的九階段，濃縮成一條可執行管線：

```
FinMind 取台股日線 → MA 交叉訊號 → 向量化回測(扣真實成本)
→ 驗證(walk-forward + Deflated Sharpe + 多重檢定示範)
→ 風控(1% 風險式部位大小) → 今日 paper 下單指令
```

再強調一次立場：這個策略（MA 交叉）**故意選一個簡單、而且大概率沒有 edge 的策略**。因為這本書要教的不是「這招會賺」，而是「**怎麼用工程方法證明一招到底有沒有用**」——這才是你老闆要你學的真本事。

目錄

1. 環境與專案結構（怎麼跑、檔案地圖）
2. 階段 1-2 資料：FinMind 抓 + 清洗（`data.py`）
3. 階段 3-4 訊號：MA 交叉 + 防未來函數（`strategy.py`）
4. 階段 5 回測：向量化 + 真實成本（`backtest.py`）
5. ★ 階段 5 的靈魂 驗證：DSR + walk-forward（`validation.py`）
6. 階段 6 風控：1% 風險式部位大小（`risk.py`）
7. 階段 7 執行：今日 paper 指令（`paper.py`）
8. 跑出來的誠實結果（怎麼解讀每個數字）
9. 怎麼把它改成「你的東西」（換策略 / 接 LLM / 接券商）
10. 結語：你已經會了最難的那一塊

1 環境與專案結構

專案在 `~/Delvin-agent/ai_trading_course/demo/`。依賴 (requests / pandas / numpy / scipy) winrig 已裝好。檔案對照《地圖》的九階段：

檔案	對應階段	做什麼
<code>config.py</code>	—	集中設定 (標的/期間/參數/成本)， 改這裡就好
<code>data.py</code>	1-2 資料/清洗	FinMind 抓台股日線 + 本地快取 + 清洗
<code>strategy.py</code>	3-4 特徵/訊號	MA 交叉訊號 (防 look-ahead) + ATR
<code>backtest.py</code>	5 回測	向量化算每日報酬，扣手續費+稅+滑價
<code>validation.py</code>	5 的靈魂	Sharpe / Deflated Sharpe / walk-forward
<code>risk.py</code>	6 風控	依停損距離反推張數，單筆風險固定 1%
<code>paper.py</code>	7 執行	把訊號+風控變成「今天該怎麼下單」
<code>run.py</code>	閉環	串起全部，印出誠實報告

工程觀念：為什麼要拆成這麼多檔？因為交易系統是「資料生產線」，每一道工序該能**獨立測試、獨立替換**。你想換策略，只動 `strategy.py`；想換成本假設，只動 `config.py`。這就是《地圖》說的「可重現、可維護」的工程紀律。

2 階段 1-2 資料：FinMind 抓 + 清洗

第一步永遠是「把乾淨、正確的資料拿進來」。data.py 做三件事：打 FinMind API、本地快取（不重複打 API）、清洗。

```
def get_prices(stock_id, start, end):
    f = CACHE / f"{stock_id}_{start}_{end}.csv"
    if f.exists():
        # 有快取就讀檔，不重打 API
        df = pd.read_csv(f, parse_dates=["date"])
    else:
        r = requests.get(API, params={"dataset": "TaiwanStockPrice",
                                      "data_id": stock_id, "start_date": start, "end_date": end})
        df = pd.DataFrame(r.json()["data"]); df.to_csv(f, index=False)
    ...
    df = df[(df["volume"] > 0) & (df["close"] > 0)] # 清洗：濾掉停牌/異常
    return df
```

這裡的工程坑（《地圖》第四部）：

- **快取很重要**：免費 API 有額度，每跑一次都重打會被限流。快取讓你可以反覆實驗。
- **FinMind 給的是「已還原權息」的價格**，所以不用自己處理除權息跳空——但你**必須知道**你的資料是哪一種，否則回測會出現假跳空。
- **免費資料沒有 point-in-time 保證、可能缺已下市股**——學習夠用，認真上線要換更可靠的源。

3 階段 3-4 訊號：MA 交叉 + 防未來函數

訊號是「把資料變成決定」。這裡用最簡單的：快線（20 日均）站上慢線（60 日均）就持有，跌破就空手。

```
def ma_cross_signal(df, fast, slow):  
    f = df["close"].rolling(fast).mean()  
    s = df["close"].rolling(slow).mean()  
    raw = (f > s).astype(int)  
    return raw.shift(1).fillna(0)          # ← 關鍵：shift(1) 防未來函數
```

這一行 `shift(1)` 是新人最常漏、最致命的防呆。沒有它，你會用「今天的收盤價」算出訊號、然後假裝「今天就能用這個訊號交易」——但收盤價要收盤才知道，你等於偷看了未來（look-ahead bias）。`shift(1)` 的意思是「昨天收盤算出的訊號，今天才進場」，這才符合現實。很多「神級回測」拿掉這一行就現形。

同檔還有 `atr()`（平均真實波幅），等一下風控設停損會用到——它衡量「這檔股票平常一天波動多大」，用來把停損設在「技術上合理」的距離，而不是隨便抓一個 %。

4 階段 5 回測：向量化 + 真實成本

回測 = 「如果當時照訊號做，會怎樣」。向量化作法：訊號 $\times$ 隔日報酬，一次算完整段歷史。

```
def run_backtest(df, signal):
    ret = df["close"].pct_change().fillna(0)
    strat_ret = signal * ret
    turn = signal.diff().abs().fillna(0)
    cost = turn * (FEE + SLIPPAGE) \
        + (signal.diff() < 0) * TAX
    return strat_ret - cost
```

有持倉才吃當日報酬
訊號改變=換手
賣出才扣證交稅
報酬要扣成本！

新人最常見的自欺：回測假設「零成本、收盤價成交」。台股一買一賣的手續費($0.1425\% \times 2$)+證交稅(0.3%)就吃掉約 0.6%，加滑價更多。**很多看起來會賺的策略，把成本加回去就翻負**——尤其進出頻繁的。這個 demo 一開始就把成本算進去，你看到的數字才誠實。

5 ★ 階段 5 的靈魂 驗證：DSR + walk-forward

這是整個 demo（也是整個工作）最重要的一章。`validation.py` 的工作是當「測謊機」——分辨一個 edge 是真的，還是你在歷史雜訊裡挖到的運氣。

5.1 Sharpe 不夠，要 Deflated Sharpe

一般 Sharpe 只看「報酬÷波動」，但它不知道你試了幾次。Deflated Sharpe (DSR) 把「你掃了幾組參數」也算進去，回頭把 Sharpe 打折：

```
def deflated_sharpe(returns, n_trials):
    sr = sharpe(returns) / sqrt(252)          # 每期 Sharpe
    sk, kt = skew(returns), kurtosis(returns) # 報酬分布的偏度/峰度
    # 你試 n_trials 次，純運氣下「最好那組」的期望 Sharpe 基準：
    sr0 = expected_max_sharpe(n_trials, T)
    # 回傳「這個 Sharpe 真的 > 純運氣基準」的機率
    return norm.cdf((sr - sr0) * sqrt(T-1) / denom)
```

白話：你掃描的參數越多（`n_trials` 越大），純運氣也能造出越高的 Sharpe，所以同樣一個 Sharpe 就越不值錢。

DSR 把這件事量化成一個 0~1 的機率：越接近 1 越可信，低於 ~0.95 就該高度懷疑。這正是你 CLAUDE.md 裡

`backtest-validation` / `quant-math` 技能在做的事。

5.2 walk-forward：只算「沒看過的年份」

```
def walk_forward(df, fast, slow):
    oos = []
    for 每個測試年份 in 訓練期之後的年份:
        sig = ma_cross_signal(df, fast, slow)[該年份]
        oos.append(run_backtest(該年份, sig)) # 只收樣本外報酬
    return concat(oos)
```

它強迫策略在「開發時沒看過」的年份反覆證明自己。**這才是真正算數的數字**——全期回測很容易因為「剛好擬合到那段歷史」而虛胖。

三個指標一起看才安全：① 全期 Sharpe（參考）② DSR（扣掉「試幾次」的真相）③ walk-forward 樣本外（沒看過的年份）。三個都好，才有資格進下一關。任何一個垮，這個策略就該死在這。

6 階段 6 風控：1% 風險式部位大小

就算策略是對的，部位開太大一樣會死。風控的核心問題是「這次該下多少」。這個 demo 用「固定風險法」：每筆交易最多只虧帳戶的 1%。

```
def position_size(entry, stop):  
    risk_amt = ACCOUNT * 0.01          # 每筆最多虧 1% = 10,000  
    per_share_risk = entry - stop      # 一股的最大虧損  
    shares = risk_amt / per_share_risk # 反推能買幾股  
    return (shares // 1000) * 1000     # 台股整張
```

反推邏輯：停損距離越遠 → 一股風險越大 → 買越少（讓總風險固定）。這就是《地圖》第六部說的「選股決定你賺多少，部位大小決定你能不能留在牌桌上」。

跑出來你會看到一個真實限制：台積電 1075 元、停損 1036，一股風險約 39 元，一整張（1000 股）風險就 39,286 元 > 你的 1% 預算（10,000）→ **shares = 0，買不起一整張**。程式會老實提示你「放寬風險% / 加大帳戶 / 改用零股」。這不是 bug，是風控的真實數學——很多人從沒算過這個，就重壓下去了。

7 階段 7 執行：今日 paper 指令

最後一步：把「訊號 + 風控」變成一張人看得懂的下單指令。paper = 模擬，不是真下單。

```
def todays_order(df):
    sig = ma_cross_signal(df, FAST, SLOW)
    holding, prev = sig.iloc[-1], sig.iloc[-2]
    if holding==1 and prev==0: action = "BUY 進場"; size = position_size(...)
    elif holding==0 and prev==1: action = "SELL 出場"
    elif holding==1:          action = "HOLD 續抱"
    else:                      action = "FLAT 空手觀望"
    return {...}
```

從 paper 到真錢的唯一正確路徑（《地圖》第四部）：回測 → paper（用真實行情、模擬資金）→ 小資金實盤 → 逐步放大。paper 會抓出回測永遠看不到的東西：滑價、延遲、API 斷線、你自己的 bug。要接真券商，就是把這張指令送進 ccxt（加密）或 Shioaji（台股）——但永遠先 paper 跑夠久。

8 跑出來的誠實結果（怎麼解讀）

以台積電（2330，2018–2024）為例，`run.py` 實際輸出：

```
策略 MA20/60（單一參數）：CAGR 0.18 Sharpe 0.92 DSR 0.99
基準 Buy & Hold          : CAGR 0.25 Sharpe 0.95 DSR 0.99
△ 掃了 47 組參數，挑最好的 MA15/60：
  天真看：Sharpe 1.065 → 看起來很神
  誠實看：DSR 0.70 ← 把「試了47次」打折後
  樣本外 walk-forward：CAGR 0.09 Sharpe 0.50
```

怎麼讀這四行（這就是工程的判斷力）：

1. 單一參數策略 **CAGR 0.18 < Buy&Hold 0.25** → 連最笨的「買了抱著」都打不過。第一關就該淘汰。
2. 掃 47 組挑出 **Sharpe 1.065** → 天真的人會興奮上線。但這是 data snooping：試夠多次總有一組好看。
3. 那組的 **DSR 掉到 0.70** → 低於 0.95 門檻，**統計上不可信**，多半是運氣。
4. **walk-forward 樣本外 Sharpe 只剩 0.50**（另一檔 2454 甚至變 -0.3）→ 沒看過的年份才是真相，edge 蒸發。

結論：這個策略沒有真 edge。而你，用工程方法「證明」了它沒有——這比盲目上線值錢一萬倍。

接你自己的經驗：這跟你 `trading_bot` 當年「Sharpe 1.58 的 pair trade、+39%/年的 funding spike」栽的跟頭一模一樣——窄窗口 + 挑最好那組 + 忘了算「試幾次」。你已經用真錢上過這堂課，現在用 30 行程式碼隨時能重演它，提醒自己別再犯。

9 怎麼把它改成「你的東西」

這個 demo 是骨架，真正的學習是你拿它去長肉。建議順序：

1 換標的與期間（5 分鐘）

改 `config.py` 的 `UNIVERSE`、`START/END`，重跑。感受不同股票、不同期間結論差多少（這本身就是一課：edge 常常 regime-dependent）。

2 換一個新策略（半天）

在 `strategy.py` 加一個新訊號函式（RSI 超賣、突破 20 日高、動能），在 `run.py` 接上去跑驗證。關鍵練習：看它過不過 DSR 與 walk-forward——大部分你想到的策略都會死在這關，這是正常的、健康的。

2 接 LLM 特徵（核心，連回《地圖》第二部）

這是你老闆最想要的方向。把「新聞情緒分數」當成額外的進場過濾條件：

```
# 概念：LLM 把新聞變成 -1~+1 情緒分數（你 MarketDaily 已有這套）
sentiment = llm_sentiment(news_for(stock_id, date)) # 文字→結構化特徵
signal = ma_cross_signal(df, 20, 60)
signal = signal & (sentiment > 0) # 只在「均線多頭 且 新聞不負面」時持有
```

但這裡是整本書的試金石：情緒到底有沒有加分，用 DSR 說話，不要用感覺。跑驗證比較「有情緒過濾 vs 沒有」的樣本外 Sharpe。如果加了沒變好甚至變差——那就誠實承認這個特徵沒用，丟掉。這就是「AI 當研究助理、人來嚴格證偽」的真實工作流。

4 接真券商 paper（進階）

把 `paper.py` 的指令字典，改送 `ccxt`（加密貨幣，你已有此技能）或 Shioaji（台股）的 paper / testnet 端點。先讓它在模擬環境跑幾週，確認沒有 API 斷線、下單邏輯、時區的 bug，再談小資金。

結語：你已經會了最難的那一塊

很多人學「AI 交易」，學的是怎麼把模型疊得更複雜。但你現在手上這個 demo 教你的，是這一行業真正稀缺的能力——**誠實地證明一個 edge 到底是不是真的**。

你現在能做到的（第一天上班就能展示）：

1. 講清楚一筆交易從資料到下單的完整流程，AI 該插哪、不該插哪。
2. 拿任何「會賺的策略」來，用 walk-forward + DSR + 真實成本把它驗一遍，當場判斷它是真 edge 還是運氣。
3. 把 LLM 放在對的位置（讀文件、生特徵、做研究），而不是叫它預測漲跌。
4. 用一個可跑的閉環當底，逐步長成你公司真正要的工具。

下一步，就是動手把第 9 章的練習做過一遍——尤其是「接 LLM 情緒特徵 + 用 DSR 驗證它有沒有用」那個。做完那個，你就真的入門了「讓 AI 幫忙交易」的工程。有問題隨時問我。慢慢來，比較快。